

Unit 3: Types of Keys & Data Integrity

3.1. Keys: Super Key, Candidate Key, Primary Key, Alternate Key, Foreign Key

Different Types of SQL Keys

A key is a single or combination of multiple fields in a table. It is used to fetch or retrieve records/data-rows from data table according to the condition/requirement. Keys are also used to create relationship among different database tables or views.

Types of SQL Keys

We have following types of keys in SQL which are used to fetch records from tables and to make relationship among tables or views.

1. Super Key

Super key is a set of one or more than one keys that can be used to identify a record uniquely in a table. **Example** : Primary key, Unique key, Alternate key are subset of Super Keys.

2. Candidate Key

A Candidate Key is a set of one or more fields/columns that can identify a record uniquely in a table. There can be multiple Candidate Keys in one table. Each Candidate Key can work as Primary Key.

Example: In below diagram ID, RollNo and EnrollNo are Candidate Keys since all these three fields can be work as Primary Key.

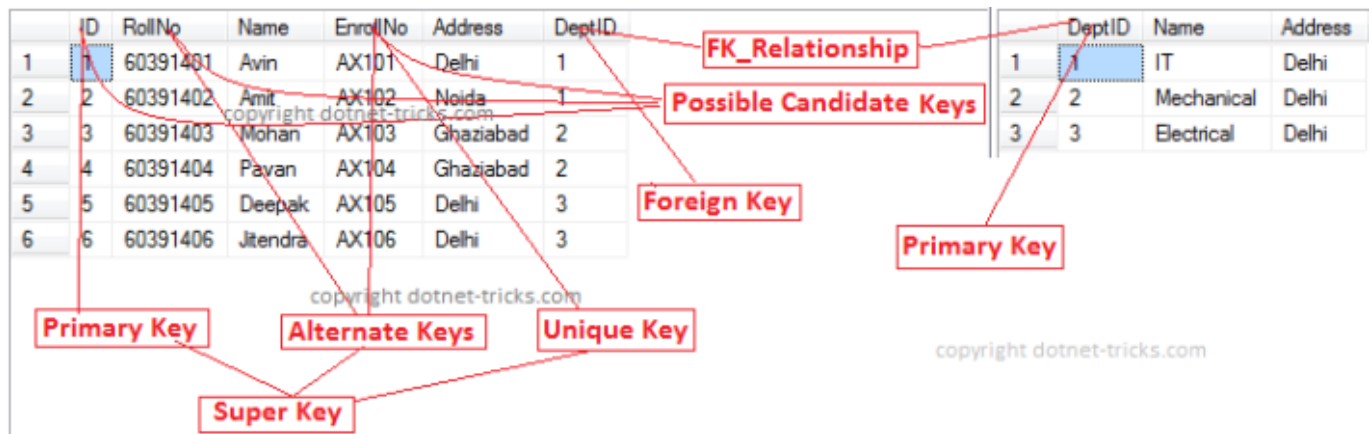
3. Primary Key

Primary key is a set of one or more fields/columns of a table that uniquely identify a record in database table. It can not accept null, duplicate values. Only one Candidate Key can be Primary Key.

4. Alternate key

An Alternate key is a key that can be work as a primary key. Basically it is a candidate key that currently is not primary key.

Example: In below diagram RollNo and EnrollNo becomes Alternate Keys when we define ID as Primary Key.



5. Composite/Compound Key

Composite Key is a combination of more than one fields/columns of a table. It can be a Candidate key, Primary key.

6. Unique Key

Unique key is a set of one or more fields/columns of a table that uniquely identify a record in database table. It is like Primary key but it can accept only one null value and it cannot have duplicate values.

7. Foreign Key

Foreign Key is a field in database table that is Primary key in another table. It can accept multiple null, duplicate values.

```
--Department Table
CREATE TABLE Department
(
    DeptID int PRIMARY KEY, --primary key
    Name varchar (50) NOT NULL,
    Address varchar (200) NOT NULL
)

--Student Table
CREATE TABLE Student
(
    ID int PRIMARY KEY, --primary key
    RollNo varchar(10) NOT NULL,
    Name varchar(50) NOT NULL,
    EnrollNo varchar(50) UNIQUE, --unique key
    Address varchar(200) NOT NULL,
    DeptID int FOREIGN KEY REFERENCES Department(DeptID) --foreign key
)
```

Practically in database, we have only three types of keys Primary Key, Unique Key and Foreign Key. Other types of keys are only concepts of DBMS which you should know.

3.2. Constraints

Integrity Constraints

SQL Constraints

SQL constraints are used to specify rules for the data in a table.

Constraints are used to limit the type of data that can go into a table. This ensures the accuracy and reliability of the data in the table. If there is any violation between the constraint and the data action, the action is aborted.

Constraints can be column level or table level. Column level constraints apply to a column, and table level constraints apply to the whole table.

The following constraints are commonly used in SQL:

- **NOT NULL** - Ensures that a column cannot have a NULL value
- **UNIQUE** - Ensures that all values in a column are different
- **PRIMARY KEY** - A combination of a NOT NULL and UNIQUE. Uniquely identifies each row in a table
- **FOREIGN KEY** - Uniquely identifies a row/record in another table
- **CHECK** - Ensures that all values in a column satisfies a specific condition
- **DEFAULT** - Sets a default value for a column when no value is specified
- **INDEX** - Used to create and retrieve data from the database very quickly

SQL NOT NULL Constraint

- By default, a column can hold NULL values.
- The NOT NULL constraint enforces a column to NOT accept NULL values.
- This enforces a field to always contain a value, which means that you cannot insert a new record, or update a record without adding a value to this field.
- The following SQL ensures that the "ID", "LastName", and "FirstName" columns will NOT accept NULL values:

Example

```
CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255) NOT NULL,  
    Age int  
);
```

If the table has already been created, you can add a NOT NULL constraint to a column with the ALTER TABLE statement.

SQL UNIQUE Constraint

The UNIQUE constraint ensures that all values in a column are different.

Both the UNIQUE and PRIMARY KEY constraints provide a guarantee for uniqueness for a column or set of columns.

A PRIMARY KEY constraint automatically has a UNIQUE constraint.

However, you can have many UNIQUE constraints per table, but only one PRIMARY KEY constraint per table.

```
CREATE TABLE Persons (  
    ID int NOT NULL UNIQUE,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int  
);
```

SQL PRIMARY KEY Constraint

The PRIMARY KEY constraint uniquely identifies each record in a database table.

Primary keys must contain UNIQUE values, and cannot contain NULL values.

A table can have only one primary key, which may consist of single or multiple fields.

```
CREATE TABLE Persons (  
    ID int NOT NULL PRIMARY KEY,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int  
);
```

DROP a PRIMARY KEY Constraint

To drop a PRIMARY KEY constraint, use the following SQL:

```
ALTER TABLE Persons  
DROP PRIMARY KEY;
```

SQL FOREIGN KEY Constraint

A FOREIGN KEY is a key used to link two tables together.

A FOREIGN KEY is a field (or collection of fields) in one table that refers to the PRIMARY KEY in another table.

The table containing the foreign key is called the child table, and the table containing the candidate key is called the referenced or parent table.

Look at the following two tables:

"Persons" table:

PersonID	LastName	FirstName	Age
1	Hansen	Ola	30
2	Svendson	Tove	23
3	Pettersen	Kari	20

"Orders" table:

OrderID	OrderNumber	PersonID
1	77895	3
2	44678	3
3	22456	2
4	24562	1

Notice that the "PersonID" column in the "Orders" table points to the "PersonID" column in the "Persons" table.

The "PersonID" column in the "Persons" table is the PRIMARY KEY in the "Persons" table.

The "PersonID" column in the "Orders" table is a FOREIGN KEY in the "Orders" table.

The FOREIGN KEY constraint is used to prevent actions that would destroy links between tables.

The FOREIGN KEY constraint also prevents invalid data from being inserted into the foreign key column, because it has to be one of the values contained in the table it points to.

SQL FOREIGN KEY on CREATE TABLE

The following SQL creates a FOREIGN KEY on the "PersonID" column when the "Orders" table is created:

```
CREATE TABLE Orders (  
    OrderID int NOT NULL,  
    OrderNumber int NOT NULL,  
    PersonID int,  
    PRIMARY KEY (OrderID),  
    FOREIGN KEY (PersonID) REFERENCES Persons(PersonID)  
);
```

SQL FOREIGN KEY on ALTER TABLE

To create a FOREIGN KEY constraint on the "PersonID" column when the "Orders" table is already created, use the following SQL:

```
ALTER TABLE Orders  
ADD FOREIGN KEY (PersonID) REFERENCES Persons(PersonID);
```

There are the domain integrity, the entity integrity, the referential integrity and the foreign key integrity constraints.

Domain Integrity

Domain integrity means the definition of a valid set of values for an attribute. You define

- data type,
 - length or size
 - is null value allowed
 - is the value unique or not
- for an attribute.

You may also define the default value, the range (values in between) and/or specific values for the attribute. Some DBMS allow you to define the output format and/or input mask for the attribute.

These definitions ensure that a specific attribute will have a right and proper value in the database.

Entity Integrity Constraint

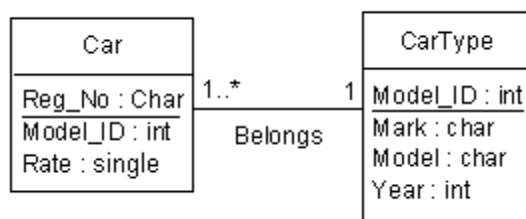
The entity integrity constraint states that primary keys can't be null. There must be a proper value in the primary key field.

This is because the primary key value is used to identify individual rows in a table. If there were null values for primary keys, it would mean that we could not identify those rows.

On the other hand, there can be null values other than primary key fields. Null value means that one doesn't know the value for that field. Null value is different from zero value or space.

In the Car Rental database in the Car table each car must have a proper and unique Reg_No. There might be a car whose rate is unknown - maybe the car is broken or it is brand new - i.e. the Rate field has a null value. See the picture below.

The entity integrity constraints assure that a specific row in a table can be identified.



Car	reg_no	model_id	rate
	ABC-112	1	45,00 €
	ABC-122	1	45,00 €
	ABC-123	1	47,00 €
	ACC-223	6	65,00 €
	ACC-224	6	65,00 €
	ACC-667	2	57,00 €
	BAA-441	5	35,00 €
	BAA-442	5	35,00 €
	BSA-224	3	45,00 €
	CCE-325	4	
	CCE-326	4	61,00 €
	CCE-327	4	62,00 €

Car Type	model_id	mark	model	year
	1	Ford	Focus	2004
	2	Ford	Mondeo	2005
	3	Peugeot	307	2004
	4	Peugeot	407	2005
	5	Renault	Clio	2004
	6	Renault	Laguna	2003

← null value

Picture. Car and CarType tables in the Rent database

Referential Integrity Constraint

The referential integrity constraint is specified between two tables and it is used to maintain the consistency among rows between the two tables.

The rules are:

1. You can't delete a record from a primary table if matching records exist in a related table.
2. You can't change a primary key value in the primary table if that record has related records.
3. You can't enter a value in the foreign key field of the related table that doesn't exist in the primary key of the primary table.
4. However, you can enter a Null value in the foreign key, specifying that the records are unrelated.

Examples

Rule 1. You can't delete any of the rows in the CarType table that are visible in the picture since all the car types are in use in the Car table.

Rule 2. You can't change any of the model_ids in the CarType table since all the car types are in use in the Car table.

Rule 3. The values that you can enter in the model_id field in the Car table must be in the model_id field in the CarType table.

Rule 4. The model_id field in the Car table can have a null value which means that the car type of that car is not known

Foreign Key Integrity Constraint

There are two foreign key integrity constraints: cascade update related fields and cascade delete related rows. These constraints affect the referential integrity constraint.

Prime and Non Prime Attributes in DBMS with Example –

- **Prime Attributes** – Attribute set that belongs to any candidate key are called Prime Attributes.
(union of all the candidate key attribute)
 $\{CK1 \cup CK2 \cup CK3 \cup \dots\}$
If Prime attribute determined by other attribute set, then more than one candidate key is possible. For example,
If A is Candidate Key, and $X \rightarrow A$, then, X is also Candidate Key .
- **Non Prime Attribute** – Attribute set does not belongs to any candidate key are called Non Prime Attributes.

Given a relation R(ABCDE) having FDs $\{A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$ Identify the prime attributes and non prime attributes.

Solution :

$(A)^+ : \{ABCDE\} \Rightarrow \text{(Candidate Key)}$

$(E)^+ : \{ABCDE\} \Rightarrow \text{(Candidate Key)}$

$\Rightarrow \text{Candidate Keys } \{A, E\}$

$\Rightarrow \text{Prime Attributes } \{A, E\}$

$\Rightarrow \text{Non Prime Attributes } \{B, C, D\}$